

DALAD: Unsupervised Detection of Global and Local Anomalies in Microservice Systems

Xiangbo Tian¹, Shi Ying¹, Tiangang Li¹, Ting Zhang², and Yong Wang²

Abstract—Accurate anomaly detection is crucial for the reliability of microservice systems. However, most existing anomaly detection approaches only detect deviations from the global expected patterns, while overlooking anomalies that conform to the global expected patterns but deviate from the local expected patterns. In this paper, we propose DALAD, a novel distribution-adversarial-learning-based anomaly detection approach for microservice systems, which jointly learns the normal and anomalous system pattern distributions to effectively detect both global and local anomalies. In detail, DALAD first designs an adversarial data generation strategy to automatically generate anomalous traces at a low cost. Then, Distribution-Adversarial-Learning Trace Representation is designed to jointly learn the multivariate-Gaussian-distribution-based vector representations of normal and anomalous traces, which can reflect the difference between traces in a more fine-grained manner. Finally, it further models the normal and anomalous system pattern distributions from these vector representations, and detects anomalies by comparing the likelihoods of traces under these distributions. Experimental results on two datasets show that DALAD achieves the best anomaly detection performance while maintaining a competitive computational cost.

Index Terms—Microservice systems, anomaly detection, trace representation, distribution adversarial learning, graph neural networks, variational autoencoder.

I. INTRODUCTION

MICROSERVICES architecture is a cloud-native architecture method, which has emerged as the mainstream method of developing cloud applications [1], [2], [3], [4], [5]. Compared with monolithic architecture, this architecture is composed of a set of services deployed independently on different machines [2], [6], which ensures the flexibility of software development and the stability of software operation.

During the operation of microservice systems, anomalies are inevitable and varied due to their complexity and large scale [7], [8], [9], [10]. The behavior patterns of microservice systems can be roughly divided into the global expected patterns and

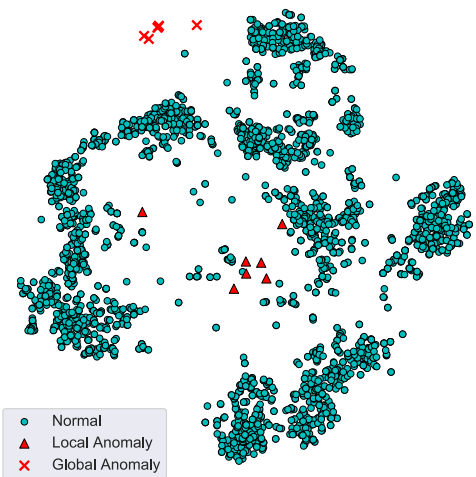


Fig. 1. Visualization of traces from the Social Network microservice system.

the local expected patterns. The former refers to the end-to-end behaviors of the entire system, such as the complete execution process of a ticket purchase workflow, while the latter represents the behaviors of a specific service, such as the response time of the payment service in the ticket purchase workflow. To intuitively illustrate different types of anomalies, we use a state-of-the-art trace representation approach iTCRL [2] to map the end-to-end execution sequences of microservice systems into a high-dimensional feature space. This feature space encodes the operation information and invocation relationships of microservice systems to achieve semantic representations for traces. In this space, the relative positions of traces reflect their similarities in the execution patterns, and traces with similar execution patterns are close to each other. For better observation, we further use t-SNE to reduce high-dimensional trace representations. This allows traces with similar features to naturally cluster in two-dimensional space since t-SNE can preserve the local structure of high-dimensional feature space and make distance reflect the similarity between traces. Fig. 1 shows the visualization of traces from the Social Network microservice system [11], [12], where green dots denote normal traces, red crosses represent traces that significantly deviate from the global expected patterns (global anomalies), and red triangles indicate traces that conform to the global expected patterns but deviate from the local expected patterns (local anomalies). These clusters intuitively show the differences in the execution patterns between traces and provide evidence of how anomalies emerge at different levels in microservice systems.

Received 22 November 2024; revised 16 December 2025; accepted 25 December 2025. Date of publication 30 December 2025; date of current version 5 February 2026. This work was supported in part by the National Key R&D Program of China under Grant 2022YFB3304300 and in part by the National Natural Science Foundation Project of China under Grant 62472329 and Grant 62072342. (Corresponding author: Shi Ying.)

Xiangbo Tian, Shi Ying, and Tiangang Li are with the School of Computer Science, Wuhan University, Wuhan 430072, China (e-mail: tianxiangbo@whu.edu.cn; yingshi@whu.edu.cn; tiangangli@whu.edu.cn).

Ting Zhang and Yong Wang are with the School of Artificial Intelligence, Wuhan Technology and Business University, Wuhan 430065, China (e-mail: zhangting@wtbu.edu.cn; wangyong001@wtbu.edu.cn).

Digital Object Identifier 10.1109/TSC.2025.3649198

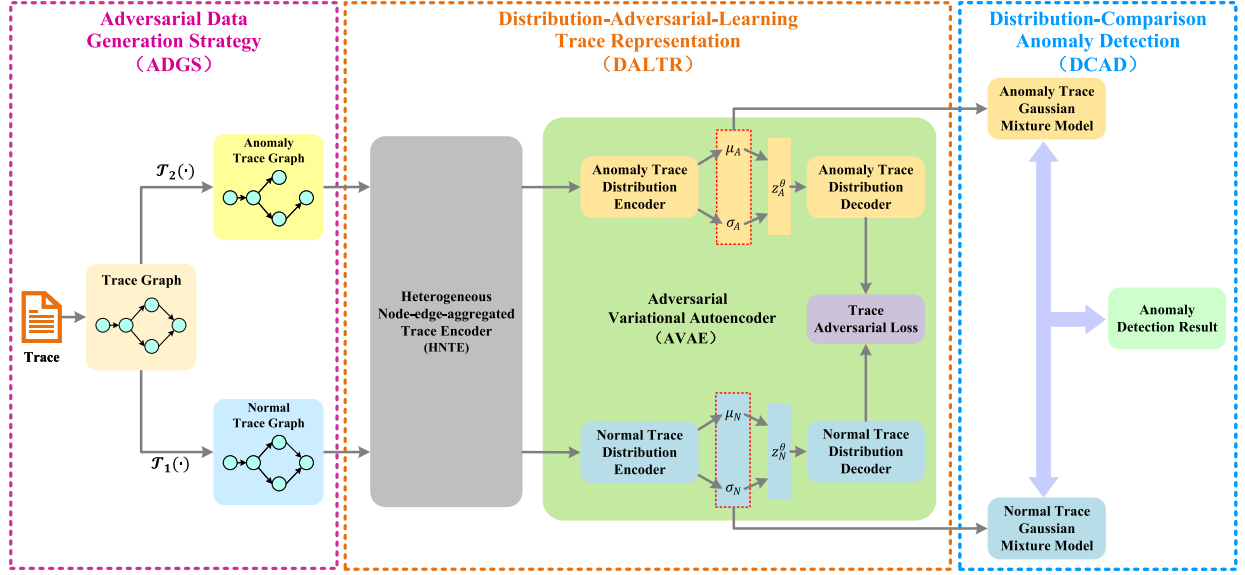


Fig. 2. Overview of DALAD.

From Fig. 1, we can observe that anomalies may occur at different levels in microservice systems, ranging from significant global deviations to subtle local deviations. However, existing anomaly detection approaches typically model only the global behavior patterns of microservice systems and detect anomalies based on deviations from such patterns [13]. Although these approaches can effectively detect global anomalies, they struggle to capture local anomalies that only deviate from the local expected patterns. Therefore, it is necessary for anomaly detection approaches in microservice systems to take both global and local anomalies into account. Motivated by this, we propose DALAD, a novel distribution-adversarial-learning-based anomaly detection approach for microservice systems, which jointly learns the normal and anomalous system pattern distributions to effectively detect both global and local anomalies.

In detail, an adversarial data generation strategy is first designed to automatically generate anomalous trace data, which can provide sufficient and various anomalous traces based on the anomalous system patterns of microservice systems at a low cost. Then, Distribution-Adversarial-Learning Trace Representation is proposed to jointly learn the multivariate-Gaussian-distribution-based vector representations of normal and anomalous traces, which can reflect the difference between traces in a more fine-grained manner. Specifically, it integrates a graph neural network with a variational autoencoder to encode each trace into a multivariate-Gaussian-distribution-based vector representation, while introducing a distribution adversarial learning strategy to jointly learn normal and anomalous traces. Unlike single-point vector representations, multivariate-Gaussian-distribution-based vector representations not only preserve the global features of traces but also capture the richer statistical and relational patterns between traces, thereby retaining subtle yet critical differences and providing a more fine-grained trace representation. Finally, DALAD learns the Gaussian mixture distributions of normal and anomalous traces from these vector

representations to capture the normal and anomalous system patterns, and detects anomalies by comparing the likelihoods of traces under these distributions. Experimental results on two datasets show that DALAD achieves the best anomaly detection performance while maintaining a competitive computational cost. In summary, our contributions are given as follows.

- To our best knowledge, we are the first to use distributions to represent traces, which can reflect the difference between traces in a more fine-grained manner.
- We design a novel distribution-adversarial-learning-based anomaly detection approach, which can effectively detect both global and local anomalies in microservice systems.
- We conduct extensive experiments on two datasets, and experimental results show that DALAD achieves the best anomaly detection performance while maintaining a competitive computational cost.

The rest of the paper is organized as follows. Section II introduces the background and preliminaries of this study. Section III describes the specific details of our proposed approach. Section IV gives the experimental results and analysis of DALAD. Section V introduces the related works. Section VI concludes our work and gives future research directions.

II. BACKGROUND AND PRELIMINARIES

A. Background

Microservice Systems: A microservice system is a large-scale distributed system. It allows developers to independently develop and deploy functional units [10] and complete user requests through these loosely-coupled functional units. This design enhances the flexibility and scalability of systems but introduces complex service interactions, which further increase operational challenges. In this context, distributed tracing plays an important role in ensuring the observability of microservice systems.

Distributed Tracing: Distributed tracing aims to capture the execution path of a user request throughout microservice systems [14]. By assigning a unique trace ID, it can effectively record the operations, service invocations and time information throughout the end-to-end execution of a user request. This design can fully preserve system observability and achieve effective monitoring for complex microservice systems.

Trace Data: A trace records a complete execution path of a user request throughout microservice systems [14]. It is composed of many spans. Each span represents a service invocation, which consists of much key execution information, such as timestamps, span contexts, span attributes, span events, span status and invocation relationships [14]. Thus, a trace can be naturally described as a graph, which forms the basis for applying graph neural networks to trace analysis.

B. Preliminaries

Graph Neural Network (GNN): Graph neural networks can effectively model graph-structured data, such as social networks and recommendation systems. By iteratively aggregating information from neighboring nodes, it can effectively learn low-dimensional vector representations for nodes or graphs, which makes it particularly suitable for modeling trace data. Popular graph neural network architectures contain Graph Convolutional Network [15], Graph Attention Network [16], and Heterogeneous Attention Network [17], etc.

Variational Autoencoder (VAE): A variational autoencoder is a generative model that learns a latent probabilistic representation by mapping input data to a latent distribution [18]. It is composed of an encoder and a decoder. The former maps input data to a latent distribution, while the latter reconstructs the original input data from the latent distribution. It can effectively capture the complex data distribution characteristics, which makes it particularly suitable for the fine-grained representation learning of traces.

Gaussian Mixture Model (GMM): A Gaussian mixture model is a probabilistic model that represents a distribution as a weighted combination of multiple Gaussian components. Each Gaussian component is defined by a mean vector and a standard deviation vector. It can effectively model complex multimodal distributions, which makes it particularly suitable for the fine-grained learning of system patterns.

III. APPROACH

The objective of DALAD is to accurately detect both global and local anomalies in microservice systems. This approach takes trace data as input and jointly learns the Gaussian mixture distributions of normal and anomalous traces to capture the normal and anomalous system patterns. During inference, it calculates and compares the likelihoods of an unseen trace under these distributions to determine whether it is anomalous.

The overview of DALAD is shown in Fig. 2, which consists of three components. **Adversarial Data Generation Strategy (ADGS)** automatically produces anomalous traces according to the anomalous system patterns of microservice systems. **Distribution-Adversarial-Learning Trace Representation**

(DALTR) jointly learns the multivariate-Gaussian-distribution-based vector representations of each normal trace and its corresponding anomalous trace. **Distribution-Comparison Anomaly Detection (DCAD)** learns the Gaussian mixture distributions of normal and anomalous traces from these vector representations, and detects anomalies by comparing the likelihoods of traces under these distributions.

A. Adversarial Data Generation Strategy

In microservice systems, trace data consists of much useful information, such as span events, span attributes and timestamps [2]. The information is very helpful for the anomaly detection task of microservice systems. However, most existing research only considers service invocation relationships but ignores the fine-grained execution information. To make full use of this information, we transform trace data into Service-Event-based Trace Graphs (SETGs) following [2]. The specific process consists of four steps:

- **Trace Data Preprocessing:** We extract Service Runtime Events (SREs), Service Invocation Events (SIEs) from traces, and calculate Service Event Metrics (SEMs) based on the span information.
- **Span Data Parsing:** We extract the constant templates and variable templates from Service Runtime/Invocation Events using the Drain algorithm [19], and divide Service Event Metrics into Service Event Runtime Metrics (SERMs) and Service Event Invocation Metrics (SEIMs) according to their meaning and effects.
- **Vector Representation:** For Service Event Runtime/Invocation Metrics, we directly transform them into vector representations, where each dimension represents a metric. For Service Runtime/Invocation Events, their templates are embedded as vector representations using GloVe [20] with TF-IDF [21].
- **Graph Construction:** We build a graph for each trace where nodes represent Service Runtime/Invocation Events described by their vectors, and edges denote the invocation relationships between service events.

The detailed definitions and processes strictly follow [2].

During the operation of microservice systems, anomalies are inevitable [9]. Among these anomalies, some deviate from the global expected patterns while others only deviate from the local expected patterns. To address this problem, DALAD simultaneously models normal and anomalous system behaviors. However, the cost of obtaining anomalous traces is very high. Therefore, we analyze the anomalous system patterns of industrial microservice systems and design Adversarial Data Generation Strategy (ADGS) to automatically generate anomalous traces. The specific generation process is given by the following equations.

$$\mathcal{G}_t^N = \mathcal{T}_1(\mathcal{G}_t) \quad (1)$$

$$\mathcal{G}_t^A = \mathcal{T}_2(\mathcal{G}_t) \quad (2)$$

where $\mathcal{G}_t$ is the SETG of trace t ; $\mathcal{G}_t^N$ and $\mathcal{G}_t^A$ represent the normal and anomalous SETGs corresponding to trace t , respectively;

$\mathcal{T}_1(\cdot)$ is the normal trace generation operation, which is the identity transformation in this paper; $\mathcal{T}_2(\cdot)$ denotes the anomalous trace generation operation, which simulates five types of anomalies in industrial microservice systems, as described below.

- *Operation Anomaly*: Randomly change the attributes of a certain proportion of span events in a trace, including event type (EventType), event label (EventLabel) and event parameters, to simulate inconsistencies in service execution events.
- *Service Metric Anomaly*: Randomly change the timestamp information of a certain proportion of traces to modify the service-level metrics derived from traces, including throughput, response time and error rate, to simulate degraded or anomalous service performance.
- *Invocation Metric Anomaly*: Randomly change a certain proportion of inter-service invocation attributes, including invocation latency and success rate, to simulate degraded or failed service invocations.
- *Operation Sequence Anomaly*: Randomly swap the execution order of a certain proportion of span events, including ParentSpanID, SpanID and startTime, to simulate incorrect execution sequences.
- *Invocation Interruption Anomaly*: Randomly remove a certain proportion of invocation relationships and all downstream span events in a trace to simulate the interruption of service invocation chains.

The modification proportions are determined according to the granularity and structural impact of different anomaly types. Operation Anomalies directly perturb fine-grained execution attributes of span events and an excessively high proportion may severely distort the execution semantics. Therefore, the modification proportion for them is set to 0.1. In contrast, other anomalies mainly affect service-level behaviors or structural attributes. For them, a proportion of 0.2 is adopted to ensure that anomalous patterns are sufficiently observable while preserving the overall structure. By the above strategy, we can generate different anomalous traces for each normal trace, which can provide sufficient and various anomalous traces at a low cost.

B. Distribution-Adversarial-Learning Trace Representation

After transforming traces into SETGs, we need to represent them by low-dimensional vectors to facilitate the subsequent anomaly detection task. To reflect the difference between traces in a more fine-grained manner, **Distribution-Adversarial-Learning Trace Representation (DALTR)** is designed to jointly learn the multivariate-Gaussian-distribution-based vector representations of each normal trace and its corresponding anomalous trace. In detail, it consists of **Heterogeneous Node-edge-aggregated Trace Encoder (HNTE)** and **Adversarial Variational Autoencoder (AVAE)**. Among them, HNTE is used to aggregate the operation and invocation information of traces, and AVAE is used to jointly learn the multivariate-Gaussian-distribution-based vector representations of traces.

1) *Heterogeneous Node-Edge-Aggregated Trace Encoder*: HNTE is used to aggregate the operation and invocation

information of traces and get their single-point vector representations. It consists of five steps: **Trace Content Initialization**, **Trace Content Transformation**, **Neighborhood Aggregation**, **Node Content Update** and **Trace Graph Pooling**.

Trace Content Initialization aims to integrate the diverse attributes of nodes in SETGs to facilitate downstream feature extraction and analysis. Since each node contains multiple attributes with different semantic meanings, an attribute-based fusion operation is applied for each node, which is shown by the following equation.

$$\mathbf{h}_i^{(0)} = \tanh(\mathbf{W}_I [\mathbf{X}_{i,0} \parallel \mathbf{X}_{i,1} \parallel \cdots \parallel \mathbf{X}_{i,n}] + \mathbf{b}_I) \quad (3)$$

where $\mathbf{h}_i^{(0)}$ is the initialized feature vector of node i ; $\mathbf{W}_I$ and $\mathbf{b}_I$ represent the learnable weight matrix and bias matrix, respectively; $\parallel$ is the concatenation operation; $\mathbf{X}_{i,k}$ denotes the k -th attribute feature vector of node i ; n represents the attribute number of node i . In SETGs, edges are described by both types and attributes, which record different useful information. For edge types, a type-mapping operation is used to map edge types to feature vectors, which is given by the following equation.

$$\mathbf{A}_{ji}^{(0)} = \text{LeakyReLU}(q(\mathbf{A}_{ji})) \quad (4)$$

where $\mathbf{A}_{ji}^{(0)}$ is the initialized feature vector of the edge type from node j to node i ; $\mathbf{A}_{ji}$ denotes the edge type from node j to node i ; $q(\cdot)$ represents a learnable feature dictionary. For edge attributes, the input attribute feature vector is directly used as the initialized edge attribute feature vector, which is given by the following equation.

$$\mathbf{E}_{ji}^{(0)} = \mathbf{E}_{ji} \quad (5)$$

where $\mathbf{E}_{ji}^{(0)}$ represents the initialized edge attribute feature vector from node j to node i ; $\mathbf{E}_{ji}$ denotes the input edge attribute feature vector from node j to node i .

Trace Content Transformation aims to project trace content into a new feature space to extract useful features and enhance feature expression capability. For nodes, a node-type-specific linear transformation is applied to project different types of nodes into the same feature space. The specific transformation process of nodes is shown by the following equation.

$$\mathbf{h}_i^{(l+1)} = \mathbf{W}_{\varphi(i)}^{(l+1)} \mathbf{h}_i^{(l)} + \mathbf{b}_{\varphi(i)}^{(l+1)} \quad (6)$$

where $\mathbf{h}_i^{(l+1)}$ is the projected feature vector of node i in layer $l+1$; $\mathbf{W}_{\varphi(i)}^{(l+1)}$ and $\mathbf{b}_{\varphi(i)}^{(l+1)}$ are the learnable weight matrix and bias matrix in layer $l+1$, respectively; $\varphi(i)$ represents the type of node i . For edges, two linear transformations are applied for their type feature vector and attribute feature vector, respectively. The specific transformation process of edges is given by the following equations.

$$\mathbf{A}_{ji}^{(l+1)} = \text{LeakyReLU}(\mathbf{W}_A^{(l+1)} \mathbf{A}_{ji}^{(l)} + \mathbf{b}_A^{(l+1)}) \quad (7)$$

$$\mathbf{E}_{ji}^{(l+1)} = \text{LeakyReLU}(\mathbf{W}_E^{(l+1)} \mathbf{E}_{ji}^{(l)} + \mathbf{b}_E^{(l+1)}) \quad (8)$$

where $\mathbf{A}_{ji}^{(l+1)}$ and $\mathbf{E}_{ji}^{(l+1)}$ are the projected edge type feature vector and edge attribute feature vector from node j to node i in

layer $l + 1$, respectively; $\mathbf{W}_A^{(l+1)}$ and $\mathbf{W}_E^{(l+1)}$ are two learnable weight matrixes in layer $l + 1$; $\mathbf{b}_A^{(l+1)}$ and $\mathbf{b}_E^{(l+1)}$ denote two learnable bias matrixes in layer $l + 1$.

Neighborhood Aggregation aims to integrate the operation and invocation information from neighboring nodes. In the aggregation process, the multi-head attention mechanism [22] is used to stabilize the attention learning process and enrich the model capacity [16]. In detail, the aggregation message is first calculated by fusing the neighbor feature vector, edge type feature vector and edge attribute feature vector. The specific message calculation process is shown by the following equation.

$$\mathbf{m}_{ji}^{(l+1,m)} = \mathbf{W}_M^{(l+1,m)} \left[\mathbf{h}_j^{(l+1)} \parallel \mathbf{A}_{ji}^{(l+1)} \parallel \mathbf{E}_{ji}^{(l+1)} \right] + \mathbf{b}_M^{(l+1,m)} \quad (9)$$

where $\mathbf{m}_{ji}^{(l+1,m)}$ is the aggregation message from node j to node i in m -th head of layer $l + 1$; $\mathbf{W}_M^{(l+1,m)}$ and $\mathbf{b}_M^{(l+1,m)}$ denote the learnable weight matrix and bias matrix in m -th head of layer $l + 1$, respectively. Then, the normalized importance weight for each neighboring node is calculated based on the node feature vector, neighbor feature vector, edge type feature vector and edge attribute feature vector to reflect the different importance of neighboring node. The specific normalization process is shown by the following equations.

$$o_{ji}^{(l+1,m)} = \text{LeakyReLU}(\mathbf{W}_O^{(l+1,m)} \cdot [\mathbf{h}_i^{(l+1)} \parallel \mathbf{h}_j^{(l+1)} \parallel \mathbf{A}_{ji}^{(l+1)} \parallel \mathbf{E}_{ji}^{(l+1)}]) \quad (10)$$

$$\alpha_{ji}^{(l+1,m)} = \frac{\exp(o_{ji}^{(l+1,m)})}{\sum_{k \in \varepsilon(i)} \exp(o_{ki}^{(l+1,m)})} \quad (11)$$

where $o_{ji}^{(l+1,m)}$ and $\alpha_{ji}^{(l+1,m)}$ are the importance weight and normalized weight from node j to node i in m -th head of layer $l + 1$, respectively. $\mathbf{W}_O^{(l+1,m)}$ represents the learnable weight matrix; $\varepsilon(i)$ denotes the neighborhood set of node i . Finally, we perform the weighted summation on all aggregation message by the following equation.

$$\mathbf{h}_i^{(l+1,m)} = \sum_{j \in \varepsilon(i)} \alpha_{ji}^{(l+1,m)} \times \mathbf{m}_{ji}^{(l+1,m)} \quad (12)$$

where $\mathbf{h}_i^{(l+1,m)}$ is the neighborhood aggregation result of node i in m -th head of layer $l + 1$.

Node Content Update aims to concatenate the neighborhood aggregation results of M attention heads of node i , and update the feature vector of node i in layer $l + 1$ based on shortcuts [23]. The specific update process is given by the following equation.

$$\mathbf{h}_i^{(l+1)} = \mathbf{h}_i^{(l)} + \bigparallel_{m=1}^M \mathbf{h}_i^{(l+1,m)} \quad (13)$$

where $\mathbf{h}_i^{(l+1)}$ is the feature vector of node i in layer $l + 1$.

Trace Graph Pooling aims to calculate the single-point vector representations of traces based on the feature vectors of all nodes in traces. In this process, the attention pooling is used to reflect the different importance of nodes. The specific pooling

process is given by the following equation.

$$\mathbf{H}_t = \sum_{i \in V(t)} \tanh(\mathbf{W}_g[\mathbf{h}_i^{(L)} \parallel \mathbf{h}_i^{(0)}] + \mathbf{b}_g) \odot \tanh(\mathbf{W}_p[\mathbf{h}_i^{(L)} \parallel \mathbf{h}_i^{(0)}] + \mathbf{b}_p) \quad (14)$$

where $\mathbf{H}_t$ is the single-point vector representation of trace t ; $\mathbf{W}_g$ and $\mathbf{W}_p$ represent the learnable weight matrixes; $\mathbf{b}_g$ and $\mathbf{b}_p$ denote the learnable bias matrixes; $V(t)$ is the node set of trace t .

2) *Adversarial Variational Autoencoder*: Although single-point vector representations can effectively capture the global characteristics of a trace, they fail to model uncertainty and feature correlations. Thus, AVAE is designed to jointly learn the multivariate-Gaussian-distribution-based vector representations of normal and anomalous traces from their single-point vector representations. It consists of three steps: **Trace Distribution Encoder**, **Trace Distribution Decoder** and **Trace Adversarial Loss**.

Trace Distribution Encoder aims to learn the hidden multivariate Gaussian distributions of traces. First, the variational net $f(\cdot)$ is used to learn the hidden feature vector $\mathbf{z}_t^\phi$ of trace t , which is shown by the following equation.

$$\mathbf{z}_t^\phi = f(\mathbf{H}_t) \quad (15)$$

where $f(\cdot)$ is a multilayer perceptron. Then, the mean vector $\boldsymbol{\mu}_t$ and standard deviation vector $\boldsymbol{\sigma}_t$ of trace t are calculated by the following equations.

$$\boldsymbol{\mu}_t = \mathbf{W}_\mu \mathbf{z}_t^\phi + \mathbf{b}_\mu \quad (16)$$

$$\boldsymbol{\sigma}_t = \mathbf{W}_\sigma \mathbf{z}_t^\phi + \mathbf{b}_\sigma \quad (17)$$

where $\mathbf{W}_\mu$ and $\mathbf{W}_\sigma$ are two learnable weight matrixes; $\mathbf{b}_\mu$ and $\mathbf{b}_\sigma$ are two learnable bias matrixes. Finally, the mean vector $\boldsymbol{\mu}_t$ and standard deviation vector $\boldsymbol{\sigma}_t$ of trace t are concatenated to get the multivariate-Gaussian-distribution-based vector representation $\mathbf{Z}_t$ of trace t , which is given by the following equation.

$$\mathbf{Z}_t = \boldsymbol{\mu}_t \parallel \boldsymbol{\sigma}_t \quad (18)$$

Trace Distribution Decoder aims to reconstruct the single-point vector representations of traces from their hidden multivariate Gaussian distributions. First, the hidden single-point vector representation $\mathbf{z}_t^\theta$ of trace t is sampled from its hidden multivariate Gaussian distribution $\mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\sigma}_t^2)$ by the following equation.

$$\mathbf{z}_t^\theta = \boldsymbol{\mu}_t + \boldsymbol{\sigma}_t \times \epsilon \quad (19)$$

where ϵ is the value sampled from $\mathcal{N}(0, 1)$. Then, the generative net $g(\cdot)$ is used to get the reconstructed single-point vector representation $\hat{\mathbf{H}}_t$ of trace t by the following equation.

$$\hat{\mathbf{H}}_t = g(\mathbf{z}_t^\theta) \quad (20)$$

where $g(\cdot)$ denotes a multilayer perceptron.

Trace Adversarial Loss aims to achieve joint learning of normal and anomalous traces. In the joint learning process, the reconstruction loss should be as small as possible, while the

hidden distribution should be as close to the normal distribution as possible. Therefore, the reconstruction loss $\mathcal{L}_N$ and $\mathcal{L}_A$ of normal and anomalous traces are defined as follows.

$$\mathcal{L}_N = \frac{1}{|T|} \sum_{t \in T} \left[\text{MSE}(\mathbf{H}_{t,N}, \hat{\mathbf{H}}_{t,N}) + \text{KL}(\mathcal{N}(\boldsymbol{\mu}_{t,N}, \boldsymbol{\sigma}_{t,N}^2), \mathcal{N}(0, 1)) \right] \quad (21)$$

$$\mathcal{L}_A = \frac{1}{|T|} \sum_{t \in T} \left[\text{MSE}(\mathbf{H}_{t,A}, \hat{\mathbf{H}}_{t,A}) + \text{KL}(\mathcal{N}(\boldsymbol{\mu}_{t,A}, \boldsymbol{\sigma}_{t,A}^2), \mathcal{N}(0, 1)) \right] \quad (22)$$

where T is the trace set; $\text{MSE}(\cdot, \cdot)$ represents the mean-square error; $\text{KL}(\cdot, \cdot)$ denotes the Kullback-Leibler divergence; $\mathbf{H}_{t,N}$ and $\mathbf{H}_{t,A}$ represent the single-point vector representations of normal trace t and its corresponding anomalous trace, respectively; $\hat{\mathbf{H}}_{t,N}$ and $\hat{\mathbf{H}}_{t,A}$ are the reconstructed single-point vector representations of normal trace t and its corresponding anomalous trace, respectively; $\mathcal{N}(\boldsymbol{\mu}_{t,N}, \boldsymbol{\sigma}_{t,N}^2)$ and $\mathcal{N}(\boldsymbol{\mu}_{t,A}, \boldsymbol{\sigma}_{t,A}^2)$ are the hidden distributions of normal trace t and its corresponding anomalous trace, respectively. Besides, the difference between the hidden distributions of normal and anomalous traces should be as large as possible to distinguish them as much as possible. Therefore, the adversarial loss $\mathcal{L}_C$ is defined as follows.

$$\mathcal{L}_C = \text{KL} \left[\mathcal{N} \left(\frac{\sum_{t \in T} \boldsymbol{\mu}_{t,N}}{|T|}, \left(\frac{\sum_{t \in T} \boldsymbol{\sigma}_{t,N}}{|T|} \right)^2 \right), \mathcal{N} \left(\frac{\sum_{t \in T} \boldsymbol{\mu}_{t,A}}{|T|}, \left(\frac{\sum_{t \in T} \boldsymbol{\sigma}_{t,A}}{|T|} \right)^2 \right) \right] \quad (23)$$

Finally, integrating these three loss, the trace adversarial loss $\mathcal{L}$ is designed as follows.

$$\mathcal{L} = \frac{\mathcal{L}_N + \mathcal{L}_A}{\beta \mathcal{L}_C} + \frac{\lambda}{2} \sum_{l=1}^{N_l} \|\mathbf{W}^{(l)}\|_F^2 \quad (24)$$

where λ is the regularization coefficient; β represents the weight of adversarial loss $\mathcal{L}_C$; N_l denotes the weight matrix number of DALTR model; $\mathbf{W}^{(l)}$ represents the l -th weight matrix of DALTR model.

C. Distribution-Comparison Anomaly Detection

In microservice systems, each normal trace can be regarded as a sample drawn from the normal system pattern distribution, while each anomalous trace corresponds to a sample drawn from the anomalous system pattern distribution. Therefore, the normal and anomalous system pattern distributions can be modeled using the vector representations of normal and anomalous traces.

First, two GMMs are used to model the normal and anomalous system pattern distributions by the following equations.

$$\begin{cases} P_N(\mathbf{Z}_{t,N} | \boldsymbol{\Theta}_N) = \sum_{k=1}^K \omega_k^N \times \mathcal{N}(\mathbf{Z}_{t,N} | \mu_k^N, \Sigma_k^N) \\ P_A(\mathbf{Z}_{t,A} | \boldsymbol{\Theta}_A) = \sum_{k=1}^K \omega_k^A \times \mathcal{N}(\mathbf{Z}_{t,A} | \mu_k^A, \Sigma_k^A) \end{cases} \quad (25)$$

where $\boldsymbol{\Theta}_N$ and $\boldsymbol{\Theta}_A$ are the parameter set of two GMMs, which consist of the weight, mean and covariance matrix of

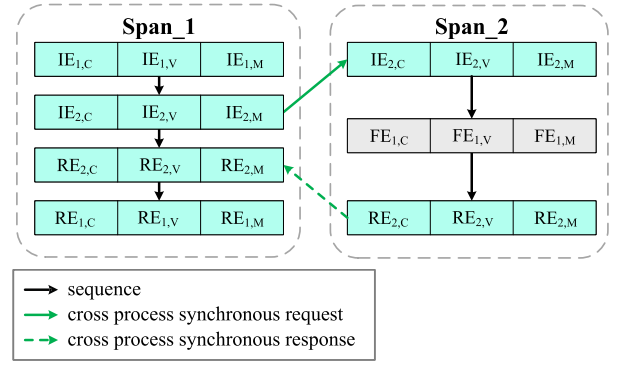


Fig. 3. SETG of the normal trace t .

each Gaussian component; ω_k^N and ω_k^A denote the weight of the k -th Gaussian component in two GMMs, respectively; K represents the number of Gaussian component in two GMMs; $\mathcal{N}(\mathbf{Z}_{t,N} | \mu_k^N, \Sigma_k^N)$ and $\mathcal{N}(\mathbf{Z}_{t,A} | \mu_k^A, \Sigma_k^A)$ are the probability density functions of the k -th Gaussian component in two GMMs, respectively. Then, the Expectation-Maximization algorithm is used to estimate the parameters $\boldsymbol{\Theta}_N$ and $\boldsymbol{\Theta}_A$ of two GMMs, respectively. Finally, DALAD quantifies the severity of anomalies by comparing the log-likelihood values of traces under normal and anomalous Gaussian mixture distributions, thereby providing a reference for alarm prioritization. Given a trace t , its anomalous severity score $S(t)$ is calculated by the following equations.

$$\log P_N(\mathbf{Z}_t | \boldsymbol{\Theta}_N) = \log \left(\sum_{k=1}^K \omega_k^N \times \mathcal{N}(\mathbf{Z}_t | \mu_k^N, \Sigma_k^N) \right) \quad (26)$$

$$\log P_A(\mathbf{Z}_t | \boldsymbol{\Theta}_A) = \log \left(\sum_{k=1}^K \omega_k^A \times \mathcal{N}(\mathbf{Z}_t | \mu_k^A, \Sigma_k^A) \right) \quad (27)$$

$$S(t) = \log P_A(\mathbf{Z}_t | \boldsymbol{\Theta}_A) - \log P_N(\mathbf{Z}_t | \boldsymbol{\Theta}_N) \quad (28)$$

where $S(t) \geq 0$ represents anomalous, $S(t) < 0$ denotes normal, and the larger the value of $S(t)$, the more severe the anomaly.

D. Running Example

To clearly illustrate the workflow of DALAD across all its components (ADGS, DALTR and DCAD), we construct a running example, including a normal trace. Table I shows the original data of this running example.

During training, given a normal trace t , ADGS first transforms it into a SETG, shown in Fig. 3, where green and gray rectangles denote SIEs and SREs, respectively, and the elements in them represent their three attributes.

Based on this graph, ADGS randomly selects one of the pre-defined anomalous trace generation operations and applies it to generate an anomalous trace. Suppose the selected operation is Operation Anomaly, an anomalous trace t' is generated, as shown in Fig. 4.

TABLE I
ORIGINAL DATA OF A RUNNING EXAMPLE

TraceID	SpanID	ParentSpanID	InstanceID	startTime (ms)	endTime (ms)	SpanEvent
t	Span_1	-	s_1	τ_1	$\tau_1 + 202$	(τ_1, IE_1) $(\tau_1 + 202, RE_1)$
	Span_2	Span_1	s_2	$\tau_1 + 50$	$\tau_1 + 180$	$(\tau_1 + 50, IE_2)$ $(\tau_1 + 120, FE_1)$ $(\tau_1 + 180, RE_2)$

Note: τ_i denote the i -th timestamp; IE_i , FE_i and RE_i denote the i -th invocation, operation and response events, respectively.

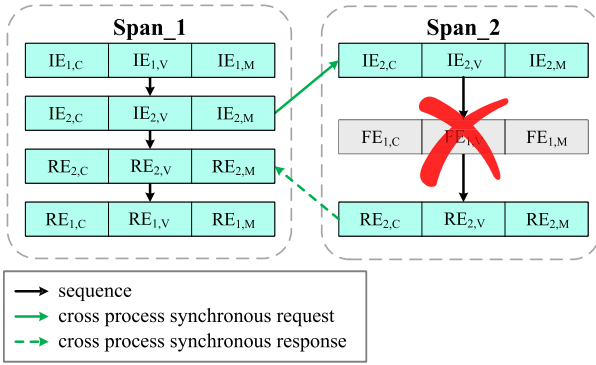


Fig. 4. SETG of the anomalous trace t' .

Then, DALTR takes the SETGs of the normal trace t and anomalous trace t' as input, and jointly learns their multivariate-Gaussian-distribution-based vector representations $\mathbf{Z}_t$ and $\mathbf{Z}_{t'}$. Finally, DCAD employs GMMs to model the normal and anomalous system pattern distributions based on $\mathbf{Z}_t$ and $\mathbf{Z}_{t'}$.

During inference, an unseen trace is first transformed into a SETG as described above. Then, this SETG is input into the trained DALTR to obtain its multivariate-Gaussian-distribution-based vector representation. Finally, this vector representation is input into the trained DCAD to calculate its anomaly severity score based on its likelihoods under the normal and anomalous system pattern distributions, and determine whether it is anomalous.

IV. EVALUATION

We implement DALAD using Python 3.7, PyTorch 1.11.0 and PyTorch Geometric 2.1.0. Based on this implementation, we conduct extensive experiments to answer the following questions.

- *RQ1*: How effective is DALAD in microservice anomaly detection compared with baselines?
- *RQ2*: What is the computational cost of DALAD in microservice anomaly detection compared with baselines?
- *RQ3*: How much does ADGS contribute to DALAD?
- *RQ4*: How much does hyperparameter configuration affect the effectiveness of DALAD?

A. Experimental Setup

1) *Datasets*: Experiments are conducted on two datasets. One is collected from the Train Ticket microservice system that

TABLE II
DETAILS OF DATASETS

Datasets	Instances	Normal Traces	Anomalous Traces	Anomaly Cases
TT	45	151911	15138	Code Defect
				Incorrect Configuration
				CPU Exhaustion
				Network Jam
SN	21	13612	1106	CPU Exhaustion
				Network Jam
				Network Loss

has been widely used in many studies [6], [8], [11], [24], [25]. The other is a public dataset provided by [11]. The details of these two datasets are shown in Table II.

- *TT*: This dataset is collected from the Train Ticket microservice system with 45 microservices [6], [8], [11], [24], [25]. We deploy this microservice system on a Kubernetes cluster composed of 6 machines with Intel Core i7-4790 3.60 GHz CPU and 8 GB RAM, and collect 151911 normal traces and 15138 anomalous traces (containing Code Defect, Incorrect Configuration, CPU Exhaustion and Network Jam).
- *SN*: This dataset is provided by [11], which is collected from the Social Network microservice system with 21 microservices [12]. It consists of 13612 normal traces and 1106 anomalous traces (containing CPU Exhaustion, Network Jam and Network Loss).

2) *Baselines*: To evaluate DALAD, five anomaly detection approaches are selected as baselines.

- *DeepLog* [26]: A log-based anomaly detection approach, which adopts an LSTM model to learn the normal sequential patterns of log events, and detects anomalies by predicting the next log event.
- *LogAnomaly* [27]: A log-based anomaly detection approach, which adopts an LSTM model to learn the normal sequential patterns and quantitative patterns of log events, and detects anomalies by predicting the next log event.
- *TraceAnomaly* [28]: A trace-based anomaly detection approach, which combines STV with a VAE with posterior flow to learn the normal system patterns from traces, and detects anomalies by identifying the traces that deviate from these patterns.

TABLE III
EFFECTIVENESS EVALUATION RESULTS OF DIFFERENT APPROACHES

Datasets	Metrics	DeepLog	LogAnomaly	TraceAnomaly	DeepTraLog	iTCRL-LOF	DALAD
TT	Precision	0.5971	0.5974	0.5046	0.4991	0.9080	1.0000
	Recall	0.9714	0.9745	0.9527	1.0000	1.0000	0.9994
	F1-Score	0.7396	0.7407	0.6597	0.6659	0.9518	0.9997
SN	Precision	0.4973	0.4973	0.7563	0.5503	0.8853	0.9944
	Recall	0.9051	0.9078	0.6736	0.8264	0.9982	0.9834
	F1-Score	0.6419	0.6426	0.7126	0.6606	0.9384	0.9888

- *DeepTraLog* [6]: A multimodal data-based anomaly detection approach, which combines GGNNs and deep SVDD to learn the normal system patterns from traces and logs, and detects anomalies by identifying the behaviors that deviate from these patterns.
- *iTCRL-LOF* [2]: A trace-based anomaly detection approach, which combines iTCRL with the LOF algorithm to learn the normal system patterns from traces, and detect anomalies by identifying the traces that deviate from these patterns.

3) *Evaluation Metrics*: To evaluate the effectiveness of DALAD, we use the following three evaluation metrics.

- *Precision*: The proportion of the real anomalous traces over all traces detected as anomalies, which is calculated as: $\text{Precision} = \frac{TP}{TP+FP}$, where TP is the number of anomalous traces detected as anomalies, and FP represents the number of normal traces detected as anomalies.
- *Recall*: The proportion of the real anomalous traces detected as anomalies over all anomalous traces, which is calculated as: $\text{Recall} = \frac{TP}{TP+FN}$, where FN denotes the number of anomalous traces detected as normal.
- *F1-Score*: The harmonic mean of Precision and Recall, which is calculated as: $\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$.

To evaluate the computational cost of DALAD, we use the following five evaluation metrics.

- *#Params*: The total number of trainable parameters in models.
- *Train Time*: The total time required to train models on the training set.
- *Infer Time*: The total time required to perform inference on the testing set.
- *Train Mem*: The peak GPU memory usage during training on the training set.
- *Infer Mem*: The peak GPU memory usage during inference on the testing set.

4) *Settings*: All experiments are conducted on a server with an Intel Core i7-12700 2.10 GHz CPU, 64 GB RAM, RTX 4080 with 16 GB GPU memory. The settings of DALAD are the following: the number of Gaussian components K is set to 32, the embedding size D is set to 32, the number of hidden layers L is set to 1, the weight of adversarial loss β is set to 1.0, the learning rate η is set to 10^{-4} , and the regularization coefficient λ is set to 10^{-4} . In experiments, the normal traces are split into training set, validation set and testing set by the ratio of 8 : 1 : 1,

while all anomalous traces are put into the testing set. To mitigate the effect of randomness, we repeat all experiments five times and report the average results. The source code and datasets of DALAD are available at <https://github.com/intse/DALAD>.

B. RQ1: Effectiveness of DALAD

Experiments are conducted on two datasets to evaluate the effectiveness of DALAD. Table III shows the effectiveness evaluation results of different approaches on two datasets. As shown in Table III, DALAD outperforms all baselines on two datasets. On the TT dataset, it improves Precision by 10.13% ~ 100.36% and F1-Score by 5.03% ~ 51.54% compared to baselines, while it is slightly worse than DeepTraLog and iTCRL-LOF in terms of Recall. On the SN dataset, it improves Precision by 12.32% ~ 99.96% and F1-Score by 5.37% ~ 54.04% compared to baselines, while it is only worse than iTCRL-LOF in terms of Recall. The excellent performance of DALAD is attributed to two reasons. (1) DALAD represents traces using multivariate-Gaussian-distribution-based vector representations, which not only preserve the global features of traces but also capture the richer statistical and relational patterns between traces, thereby retaining subtle yet critical differences and providing a more fine-grained trace representation. (2) DALAD jointly learns the normal and anomalous system pattern distributions and detects anomalies by comparing the likelihoods of traces under these distributions, which can effectively detect both global and local anomalies in microservice systems.

C. RQ2: Computational Cost of DALAD

The computational cost of anomaly detection models is crucial for the anomaly detection task in microservice systems. Table IV shows the computational cost evaluation results of different approaches on two datasets. As shown in Table IV, DALAD shows superior efficiency and resource usage on both datasets. On the TT dataset, the training time of DALAD (1671.77 s) is only 3.9% of that of LogAnomaly, and its inference time (40.16 s) is also significantly lower than most baselines. Although the parameter size and training memory of DALAD are larger than simple baselines, such as DeepLog and iTCRL-LOF, DALAD achieves better anomaly detection performance at a lower cost compared to large baselines such as TraceAnomaly (7.40 M parameters, 13607 MB Memory), LogAnomaly (0.19 M parameters, 8360 MB memory), and

TABLE IV
COMPUTATIONAL COST EVALUATION RESULTS OF DIFFERENT APPROACHES

Datasets	Metrics	DeepLog	LogAnomaly	TraceAnomaly	DeepTraLog	iTCRL-LOF	DALAD
TT	#Params (M)	0.10	0.19	7.40	0.81	0.04	0.25
	Train Time (s)	8722.00	42675.00	3127.00	14396.83	1527.16	1671.77
	Infer Time (s)	242.86	4984.51	95.13	24.50	111.06	40.16
	Train Mem (MB)	639.33	8360.23	13607.75	836.94	987.03	1623.68
	Infer Mem (MB)	470.24	751.73	200.00	594.00	734.00	642.00
SN	#Params (M)	0.06	0.12	7.18	0.81	0.04	0.24
	Train Time (s)	157.00	1234.00	336.27	2872.35	206.53	259.62
	Infer Time (s)	3.02	15.15	9.17	3.63	4.44	4.48
	Train Mem (MB)	426.06	6808.84	13594.00	185.79	702.61	727.44
	Infer Mem (MB)	258.06	286.94	200.06	86.00	72.00	68.00

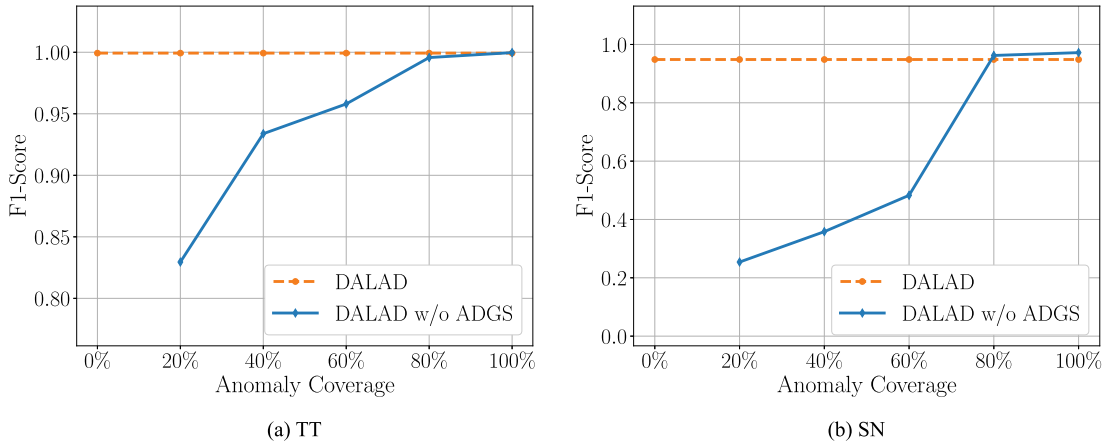


Fig. 5. Effectiveness evaluation results under different anomaly coverage.

DeepTraLog (0.81 M parameters, 836 MB memory). On the SN dataset, DALAD achieves the lowest inference memory (68 MB) while maintaining competitive training and inference times.

In summary, although DALAD integrates GNN and VAE, its computational cost is only slightly higher than that of simple baselines but significantly lower than that of large baselines by effective network depth controlling, embedding dimension controlling and mini-batch training strategy. Moreover, the computational cost of DALAD mainly lies in the training stage, and the inference cost is low, which makes it feasible to be deployed in large-scale microservice systems. In practical applications, DALAD can be efficiently trained on a single 16 GB GPU and further reduce computational cost through mixed-precision training and neighborhood sampling.

D. RQ3: Contribution of ADGS

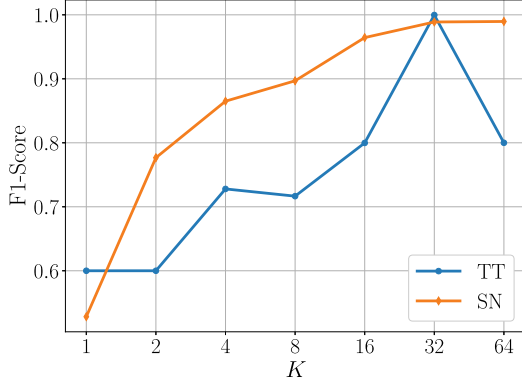
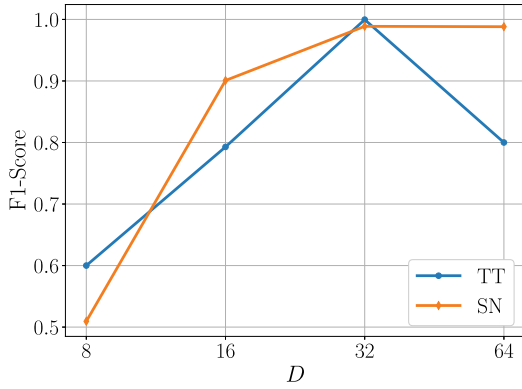
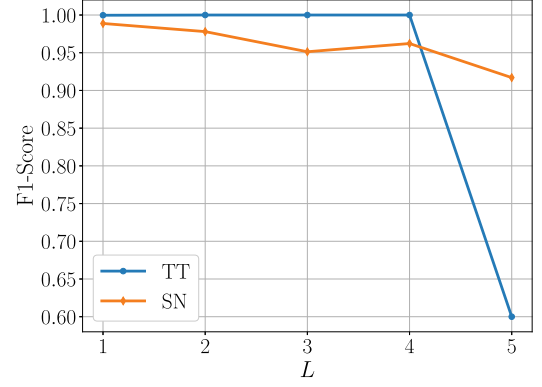
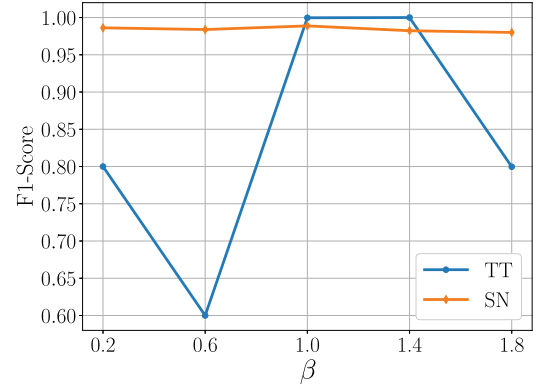
To evaluate the contribution of ADGS to DALAD, we derive a variant of DALAD by removing ADGS from DALAD, in which real anomalous traces are used for training. Fig. 5 shows the effectiveness evaluation results under different anomaly coverage. As shown in Fig. 5, DALAD significantly outperforms DALAD w/o ADGS in low-anomaly-coverage scenarios while

maintaining comparable effectiveness in high-anomaly-coverage scenarios. In two datasets, the F1-Score of DALAD w/o ADGS increases with anomaly coverage, while that of DALAD remains stable, as it does not rely on real anomalous traces. When anomaly coverage is lower than 80%, DALAD significantly outperforms DALAD w/o ADGS, while DALAD w/o ADGS approaches or even surpasses DALAD when anomaly coverage exceeds 80%. This phenomenon shows that ADGS effectively compensates for the lack of anomalous traces by automatically generating various anomalous traces. This advantage is particularly critical in real-world scenarios where anomalous traces are scarce and costly to obtain, which makes DALAD more applicable and robust in real-world scenarios.

E. RQ4: Sensitivity of Hyperparameter

The effectiveness of DALAD is affected by six hyperparameters: number of Gaussian components K , embedding size D , number of hidden layers L , weight of adversarial loss β , learning rate η , and regularization coefficient λ .

To evaluate the impact of K , experiments are conducted on two datasets when other hyperparameters are fixed. Fig. 6 shows the impact of hyperparameter K . As shown in Fig. 6, the

Fig. 6. Impact of hyperparameter K .Fig. 7. Impact of hyperparameter D .Fig. 8. Impact of hyperparameter L .Fig. 9. Impact of hyperparameter β .

F1-Score of DALAD first increases and then starts to decrease or stabilize with K , and reaches a relatively high level around $K = 32$ on both datasets. This is because DALAD needs a suitable K to capture the complex data distributions. A smaller K will lead to underfitting, while a larger K will lead to overfitting. Therefore, the choice of K is crucial to balance model complexity and generalization performance.

We explore the impact of D on two datasets when other hyperparameters are fixed. Fig. 7 gives the impact of hyperparameter D . As shown in Fig. 7, the F1-Score of DALAD first rises and then starts to drop or stabilize with D , and reaches a relatively high level around $D = 32$ on both datasets. The reason is that DALAD relies on an appropriate D to effectively encode useful information for anomaly detection. A smaller D fails to fully capture useful features, while a larger D may introduce redundant information. Therefore, the choice of D must strike a balance between information preservation and redundancy control.

We check the impact of L on two datasets when other hyperparameters are fixed. Fig. 8 shows the impact of hyperparameter L . From Fig. 8, we can see that the F1-Score of DALAD first stabilizes and then starts to decrease with L on the TT dataset while continuously decreasing on the SN dataset. It reaches a relatively high F1-Score around $L = 1$ on both datasets. This is because DALAD needs a suitable L to learn helpful information for anomaly detection, and a larger L may lead to over-smoothing.

Therefore, the choice of L is crucial to learn useful information for anomaly detection.

To evaluate the impact of β , experiments are conducted on two datasets when other hyperparameters are fixed. Fig. 9 gives the impact of hyperparameter β . As shown in Fig. 9, the F1-Score of DALAD first increases and then starts to drop with β on the TT dataset while stabilizing on the SN dataset. It reaches a relatively high F1-Score around $\beta = 1.0$ on both datasets. The reason is that β is used to control the trade-off between distribution fitting and normal-anomalous distribution separation. A smaller β will lead to excessive attention to distribution separation and reduce distribution fitting, while a larger β will weaken distribution separation and pay more attention to distribution fitting. Therefore, the choice of β must strike a balance between distribution fitting and distribution separation.

We explore the impact of η on two datasets when other hyperparameters are fixed. Fig. 10 shows the impact of hyperparameter η . As shown in Fig. 10, DALAD achieves a good performance and shows insensitivity to η on the SN dataset. On the TT dataset, DALAD performs well when η is small, but its F1-Score significantly decreases when $\eta > 10^{-4}$, which indicates that it is highly sensitive to η . It reaches a relatively high F1-Score around $\eta = 10^{-4}$ on both datasets. This is because the SN dataset has a good distribution and low noise, while the TT dataset has a complex distribution and more noise. Therefore, the choice of η is very important for ensuring DALAD convergence.

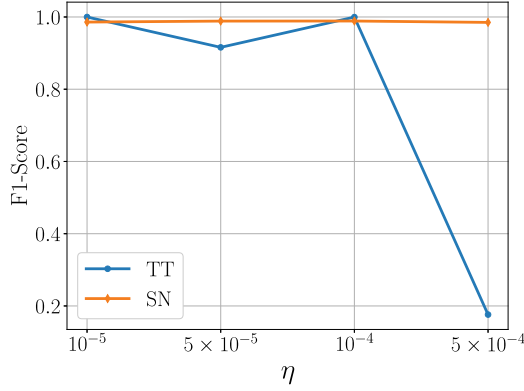


Fig. 10. Impact of hyperparameter η .

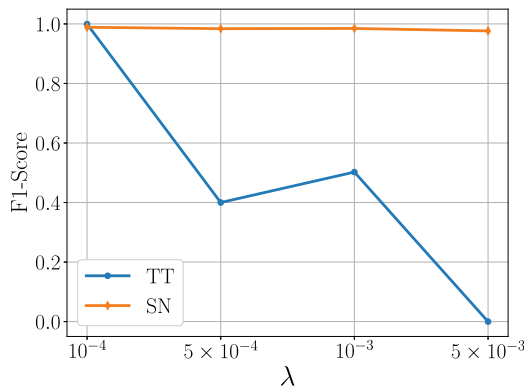


Fig. 11. Impact of hyperparameter λ .

We check the impact of λ on two datasets when other hyperparameters are fixed. Fig. 11 gives the impact of hyperparameter λ . From Fig. 11, we can observe that the F1-Score of DALAD stabilizes on the SN dataset while decreasing with λ on the TT dataset. DALAD reaches a relatively high F1-Score around $\lambda = 10^{-4}$ on both datasets. The TT dataset has a more complex distribution and more noise than the SN dataset, which makes the former more sensitive to regularization than the latter. Therefore, the choice of λ must strike a balance between distribution fitting and distribution generalization.

F. Threats to Validity

The threats to internal validity mainly lie in the collection of normal data and the design of Adversarial Data Generation Strategy (ADGS). (1) The effectiveness of DALAD is directly affected by the coverage of normal data since it relies on normal data for training. If certain types of user requests are missing from normal data, DALAD may misidentify these missing parts as anomalies. To mitigate this threat, we simulate various types of user requests during data collection to ensure the coverage of normal data. (2) ADGS aims to automatically generate anomalous trace data, which is the basis of DALAD. Therefore, its quality determines the effectiveness of DALAD. To mitigate this threat, we carefully design this strategy based on the anomalous system patterns of industrial microservice systems.

The threats to external validity mainly lie in the experimental environment. Although DALAD is designed as a model that can be trained offline and applied in an online inference manner, all experiments were conducted on two datasets collected from the widely used benchmark microservice systems, Train Ticket and Social Network. There is still a potential threat to external validity since the proposed approach has not yet been integrated into a real production system. We mitigate this threat from three aspects. First, these two datasets consist of complex traces with more than 700 and 150 events, respectively, which are comparable to the hundreds of events typically found in real production systems [6], [8]. Second, we analyze the typical anomalies in real production systems and design anomalous cases based on them, which can ensure that the anomalous scenarios used in our evaluation are representative. Finally, we plan to collaborate with a large-scale cloud provider in the future to further validate our approach.

V. RELATED WORKS

Accurate anomaly detection is crucial for ensuring the reliability of microservice systems. Existing studies use various observability data to detect anomalies and can be divided into four classes: metric-based anomaly detection approaches, log-based anomaly detection approaches, trace-based anomaly detection approaches and multimodal data-based anomaly detection approaches.

Metric-based anomaly detection approaches: These approaches usually detect anomalies by monitoring changes in service metrics. For example, Nedelkoski et al. [29] use an unsupervised deep Bayesian network to capture anomalous changes in service response time. Li et al. [30] introduce intervention recognition to identify anomalous metrics, while Yu et al. [8] apply the k - σ rule for the same purpose. Xie et al. [31] further extend this idea by designing a regression-based latent-space intervention recognition under limited observability. These approaches are simple and interpretable, but fail to detect anomalies caused by code defects or topological dependencies in microservice systems.

Log-based anomaly detection approaches: These approaches commonly focus on learning the normal sequential patterns of log events to detect anomalies as deviations from these patterns. For instance, Du et al. [26] use an LSTM model to learn the normal sequential patterns of log events and detect anomalies by predicting next log events. Meng et al. [27] extend this idea by also modeling the quantitative patterns of log events. Zhang et al. [32] represent log events through semantic embeddings and use an attention-based Bi-LSTM model to detect anomalies. These approaches are effective in detecting anomalies caused by code defects. However, they ignore the topological features of microservice systems, which limits their ability to capture inter-service dependencies.

Trace-based anomaly detection approaches: These approaches typically leverage traces to learn normal system patterns and detect anomalies. For example, Liu et al. [28] combine Service Trace Vector with a VAE with posterior flow to learn the normal system patterns from traces and detect anomalies

as those traces that deviate significantly from these patterns. Nedelkoski et al. [33] employ a multi-modal LSTM model to capture the normal sequential patterns of traces and flag deviations as anomalies. Tian et al. [2] integrate a GNN with causal intervention to detect anomalies based on data density. These approaches consider the topological features of microservice systems and can detect anomalies caused by code defects. However, they cannot detect anomalies that conform to the global expected patterns but deviate from the local expected patterns.

Multimodal data-based anomaly detection approaches: These approaches usually integrate multimodal observability data to capture normal system patterns and detect anomalies. For example, Zhang et al. [6] combine a GGNN [34] and deep SVDD [35] to detect anomalies based on the unified graph representations constructed from traces and logs. Similarly, Chen et al. [36] integrate a GNN with an LSTM model to learn the normal system patterns from the unified graphs built using traces and performance metrics of containers, and flag deviations as anomalies. By integrating multimodal observability data, these approaches can detect anomalies reflected in different observability data. However, they remain limited in detecting anomalies that conform to the global expected patterns but deviate from the local expected patterns.

VI. CONCLUSION AND FUTURE WORK

In this paper, we propose DALAD, a novel distribution-adversarial-learning-based anomaly detection approach for microservice systems. This approach jointly learns the normal and anomalous system pattern distributions to effectively detect both global and local anomalies. Specifically, we first design an adversarial data generation strategy, which can automatically generate anomalous traces at a low cost. Then, Distribution-Adversarial-Learning Trace Representation is proposed to jointly learn the multivariate-Gaussian-distribution-based vector representations of normal and anomalous traces, which can not only preserve the global features of traces but also capture the richer statistical and relational patterns between traces. Finally, DALAD learns the normal and anomalous system pattern distributions from these vector representations, and detects anomalies by comparing the likelihoods of traces under these distributions. Extensive experiments show that DALAD achieves the best anomaly detection performance while maintaining competitive computational cost. Notably, this work is the first to use distributions to represent traces and overcomes the limitations of existing approaches that rely solely on modeling the global expected patterns, which can effectively enable more robust and fine-grained anomaly detection. By jointly modeling the normal and anomalous system patterns, DALAD can effectively detect both global and local anomalies in microservice systems. Such capability can help DevOps managers to promptly detect and handle both global and local anomalies in microservice systems, thereby significantly improving system reliability and user satisfaction.

In the future, we will further explore more comprehensive and universal adversarial data generation strategies to improve the generalization capability of DALAD. Secondly, we will cooperate with a large-scale cloud provider to integrate DALAD

into real production systems and further validate its practical effectiveness and usability.

REFERENCES

- [1] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Migrating to cloud-native architectures using microservices: An experience report," in *Proc. 4th Eur. Conf. Service-Oriented Cloud Comput.*, 2015, pp. 201–215.
- [2] X. Tian et al., "iTCL: Causal-intervention-based trace contrastive representation learning for microservice systems," *IEEE Trans. Softw. Eng.*, vol. 50, no. 10, pp. 2583–2601, Oct. 2024.
- [3] M. Ma, W. Lin, D. Pan, and P. Wang, "Servicerank: Root cause identification of anomaly in large-scale microservice architectures," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3087–3100, Sep./Oct. 2022.
- [4] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey," *ACM Comput. Surv.*, vol. 55, no. 3, 2022, Art. no. 59.
- [5] Y. Gan, M. Liang, S. Dev, D. Lo, and C. Delimitrou, "Sage: Practical and scalable ML-driven performance debugging in microservices," in *Proc. 26th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2021, pp. 135–151.
- [6] C. Zhang et al., "Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 623–634.
- [7] S. Gu et al., "TrinityRCL: Multi-granular and code-level root cause localization using multiple types of telemetry data in microservice systems," *IEEE Trans. Softw. Eng.*, vol. 49, no. 5, pp. 3071–3088, May 2023.
- [8] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, "Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2023, pp. 553–565.
- [9] L. Wang, N. Zhao, J. Chen, P. Li, W. Zhang, and K. Sui, "Root-cause metric location for microservice systems via log anomaly detection," in *Proc. 2020 IEEE Int. Conf. web Serv.*, 2020, pp. 142–150.
- [10] S. Zhang et al., "Robust failure diagnosis of microservice system through multimodal data," *IEEE Trans. Serv. Comput.*, vol. 16, no. 6, pp. 3851–3864, Nov./Dec. 2023.
- [11] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.*, 2023, pp. 1750–1762.
- [12] Y. Gan et al., "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2019, pp. 3–18.
- [13] M. Panahandeh, A. Hamou-Lhadj, M. Hamdaqa, and J. Miller, "Serviceanomaly: An anomaly detection approach in microservices using distributed traces and profiling metrics," *J. Syst. Softw.*, vol. 209, 2024, Art. no. 111917.
- [14] Opentelemetry 2025 Accessed, Sep. 20, 2025 [Online]. Available: <https://opentelemetry.io/>
- [15] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, "Spectral networks and locally connected networks on graphs," in *Proc. 2nd Int. Conf. Learn. Representations*, 2014.
- [16] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. 6th Int. Conf. Learn. Representations*, 2018.
- [17] X. Wang et al., "Heterogeneous graph attention network," in *Proc. 2019 World Wide Web Conf.*, 2019, pp. 2022–2032.
- [18] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *Proc. 2nd Int. Conf. Learn. Representations*, 2014.
- [19] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. 2017 IEEE Int. Conf. Web Serv.*, 2017, pp. 33–40.
- [20] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proc. 2014 Conf. Empirical Methods Natural Lang. Process.*, 2014, pp. 1532–1543.
- [21] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.
- [22] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
- [23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. 33rd IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

- [24] C. Zhang et al., "Tracecl: Contrastive representation learning for microservice trace analysis," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2022, pp. 1221–1232.
- [25] X. Zhou et al., "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, Feb. 2021.
- [26] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.
- [27] W. Meng et al., "Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 4739–4745.
- [28] P. Liu et al., "Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks," in *Proc. 31st Int. Symp. Softw. Rel. Eng.*, 2020, pp. 48–58.
- [29] S. Nedelkoski, J. Cardoso, and O. Kao, "Anomaly detection and classification using distributed tracing and deep learning," in *Proc. 19th IEEE/ACM Int. Symp. Cluster, Cloud Grid Comput.*, 2019, pp. 241–250.
- [30] M. Li et al., "Causal inference-based root cause analysis for online service systems with intervention recognition," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 3230–3240.
- [31] Z. Xie et al., "Microservice root cause analysis with limited observability through intervention recognition in the latent space," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 6049–6060.
- [32] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2019, pp. 807–817.
- [33] S. Nedelkoski, J. Cardoso, and O. Kao, "Anomaly detection from system tracing data using multimodal deep learning," in *Proc. 12th Int. Conf. Cloud Comput.*, 2019, pp. 179–186.
- [34] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, "Gated graph sequence neural networks," in *Proc. 4th Int. Conf. Learn. Representations*, 2016.
- [35] L. Ruff et al., "Deep one-class classification," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
- [36] J. Chen et al., "Tracegra: A trace-based anomaly detection for microservice using graph deep learning," *Comput. Commun.*, vol. 204, pp. 109–117, 2023.



Shi Ying is currently a professor with the School of Computer Science, Wuhan University, Wuhan, China. His main research interests include software engineering, intelligent operation and maintenance management, cloud computing, cloud service software, and artificial intelligence.



Tiangang Li is currently working toward the PhD degree in the School of Computer Science, Wuhan University, Wuhan, China. His research interests include cloud computing, software engineering, resource management, and reinforcement learning.



Ting Zhang is currently an associate professor with the School of Artificial Intelligence, Wuhan Technology and Business University, Wuhan, China. Her research interests include intelligent software engineering, data mining, Big Data analysis technology, and artificial intelligence.



Xiangbo Tian is currently working toward the PhD degree with the School of Computer Science, Wuhan University, Wuhan, China. His current research interests include microservices and AIops.



Yong Wang is currently an associate professor with the School of Artificial Intelligence, Wuhan Technology and Business University, Wuhan, China. His research interests include intelligent software engineering, cloud computing, Big Data analysis technology, and artificial intelligence.